

| Information and Communication Technology

FACULTY OF ENGINEERING AND TECHNOLOGY
BACHELOR OF TECHNOLOGY
BIG DATA ANALYTICS LABORATORY
(303105362)

6TH SEMESTER

COMPUTER SCIENCE & ENGINEERING
DEPARTMENT

LABORATORY MANUAL

CERTIFICATE

This is to certify that PANKAJ YADAV with enrolment no.
2303031250098 has successfully completed his/her
laboratory experiments in the **BIG DATA ANALYTICS**
LABORATORY (303105362) from the Department of
COMPUTER SCIENCE AND ENGINEERING during the
academic year 2025-26.

Information and Communication Technology

INDEX

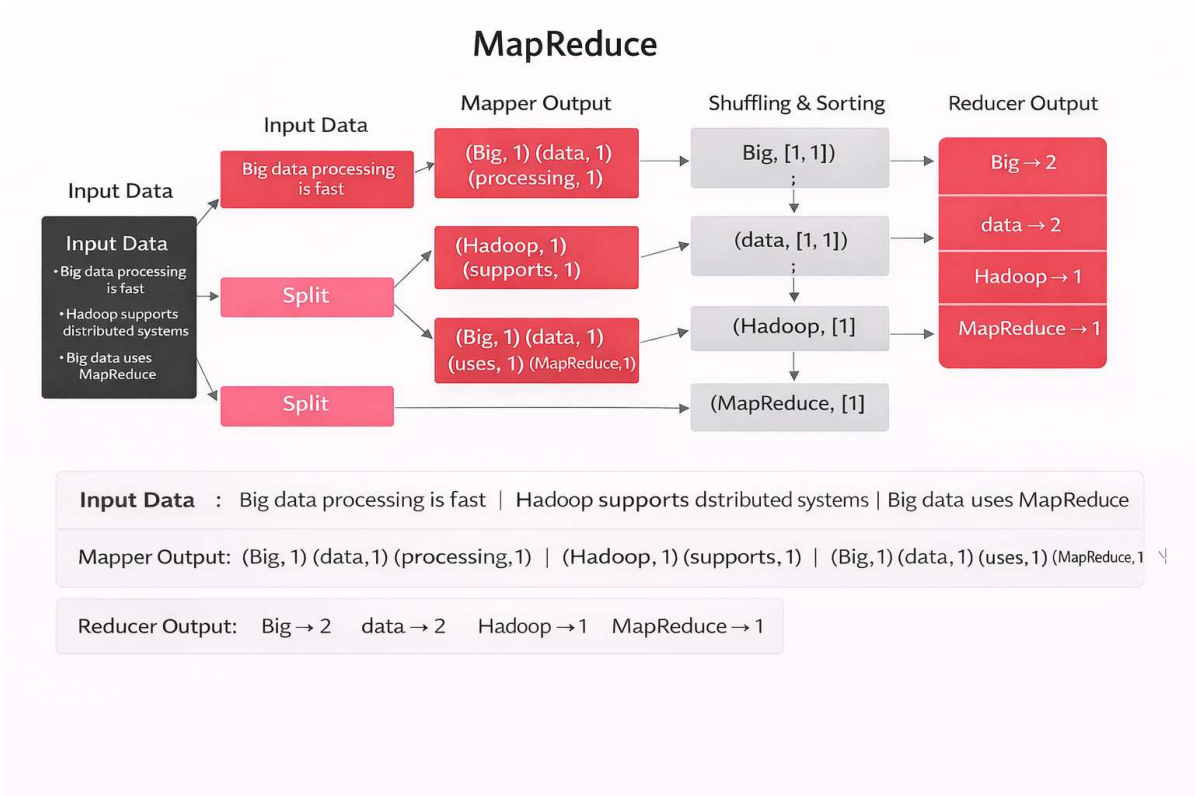
SR NO	NAME	START DATE	END DATE	MARKS	SIGN
1	To understand the overall programming architecture using Map Reduce API.				
2	Write a program of Word Count in Map Reduce over HDFS.				
3	Basic CRUD operations in MongoDB.				
4	Store the basic information about students such as roll no, name, date of birth, and address of student using various collection types such as List, Set and Map.				
5	Basic commands available for the Hadoop Distributed File System.				
6	Basic commands available for HIVE Query Language.				
7	Basic commands of HBASE Shell.				
8	Creating the HDFS tables and loading them in Hive and learn joining of tables in Hive.				

Practical -1

Aim: To Understand the overall programming Architecture using MapReduce.

- Hadoop mainly relies on two core components, **MapReduce and HDFS**, which together provide high performance and reliability.
- MapReduce is a **distributed programming framework** designed to process very large datasets by executing tasks in parallel across multiple systems.
- In this model, input data is **divided into smaller chunks**, processed independently, and then **merged** to generate the final output.
- MapReduce libraries are available in **multiple programming languages**, each offering different performance optimizations.
- The main objective of MapReduce is to **break down complex jobs into smaller tasks** and combine their results, thereby minimizing network load and reducing overall processing overhead in the cluster.
- The MapReduce workflow is primarily divided into **two stages: the Map stage and the Reduce stage**.
 1. **map(), filter(), and reduce()** are functional programming tools available in Python that follow the MapReduce concept.
 2. These functions are commonly implemented using **lambda expressions** for concise and efficient operations.

Information and Communication Technology



- **map():**

It is used to perform the same operation on all elements of a collection such as a list, tuple, or set and returns the updated values.

```

https://colab.research.google.com/#scrollTo=IT-Eod_cdqNr

Welcome To Colab  Cannot save changes
File Edit View Insert Runtime Tools Help

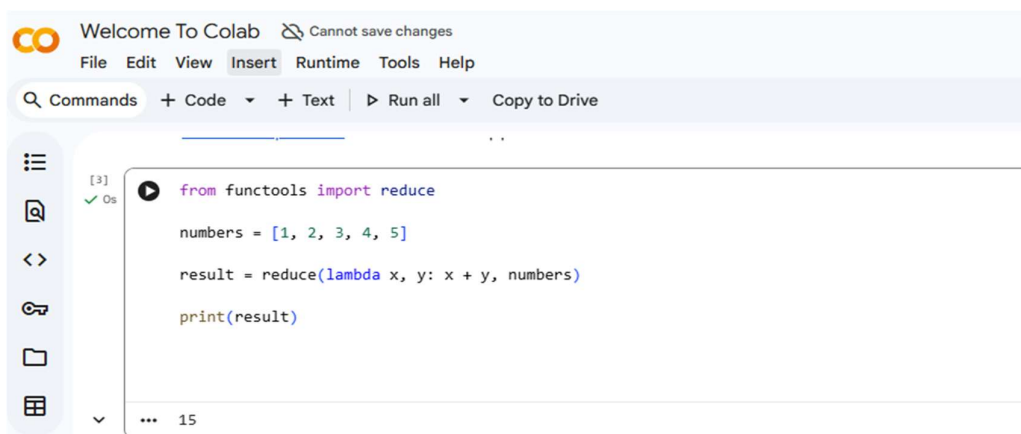
[?] Os
# map() example
items = [15, 21, 33, 64, 95]
a = list(map(lambda x: x**3, items))
print(a)

... [3375, 9261, 35937, 262144, 857375]

```

Information and Communication Technology

- `reduce()` : The `reduce()` function processes all elements of an iterable by repeatedly applying a given function and combines them into **one final output value**. It reduces a sequence of values into a single result such as a sum, product, or maximum.



The screenshot shows a Google Colab interface. At the top, it says "Welcome To Colab" and "Cannot save changes". Below that is a menu bar with "File", "Edit", "View", "Insert", "Runtime", "Tools", and "Help". A search bar contains "Commands". Below the menu bar are buttons for "+ Code", "+ Text", "Run all", and "Copy to Drive". On the left side, there is a sidebar with icons for file management. The main code area shows the following code:

```
[3] ✓ Os
from functools import reduce

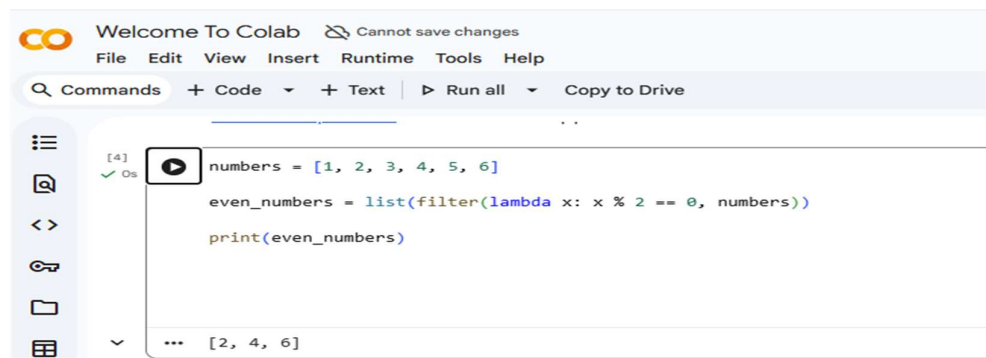
numbers = [1, 2, 3, 4, 5]

result = reduce(lambda x, y: x + y, numbers)

print(result)
```

The output at the bottom is "... 15".

- `filter()` : The `filter()` function in Python evaluates each element of an iterable using a given condition and returns only those elements for which the condition is **True**.



The screenshot shows a Google Colab interface. At the top, it says "Welcome To Colab" and "Cannot save changes". Below that is a menu bar with "File", "Edit", "View", "Insert", "Runtime", "Tools", and "Help". A search bar contains "Commands". Below the menu bar are buttons for "+ Code", "+ Text", "Run all", and "Copy to Drive". On the left side, there is a sidebar with icons for file management. The main code area shows the following code:

```
[4] ✓ Os
numbers = [1, 2, 3, 4, 5, 6]

even_numbers = list(filter(lambda x: x % 2 == 0, numbers))

print(even_numbers)
```

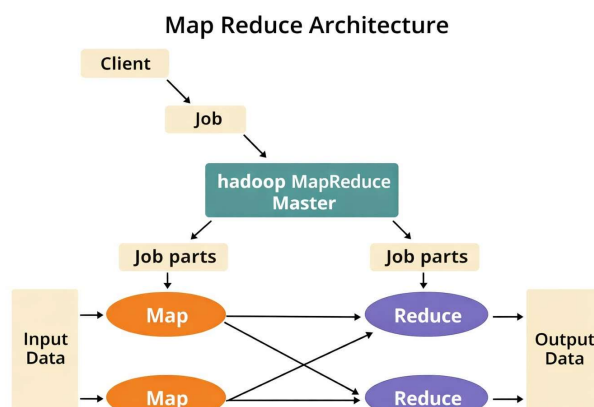
The output at the bottom is "... [2, 4, 6]".

Practical: 2

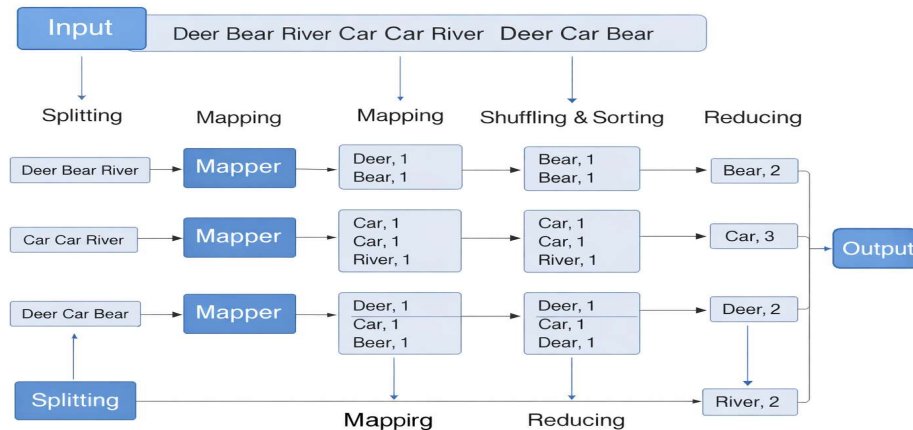
- **Aim: Write a program of Word Count in Map Reduce over HDFS.**

Description: MapReduce is a distributed computing model designed to handle very large volumes of data by dividing the work across many interconnected computers called nodes. These nodes together form a cluster, where data is commonly stored in a distributed storage system such as HDFS. Instead of moving data to a central processor, the computation is moved to the location where the data already exists, improving efficiency and performance.

In MapReduce, the original input data is divided into smaller, independent chunks so that multiple tasks can run simultaneously. Each chunk is transformed into key–value pairs, which are then processed by the Mapper function. The Mapper applies user-defined logic to each input pair and generates intermediate key–value pairs. These intermediate pairs may differ in type and structure from the original input.



The output produced by the Mapper is forwarded to the Reducer phase. Before reduction, the system automatically groups and sorts the intermediate key–value pairs based on their keys. The Reducer function then processes each group, applies aggregation or summarization logic, and generates the final output. The completed results are stored back into HDFS in the required format.



MapReduce is a programming model designed for processing large amounts of data in parallel across a distributed cluster of computers. Here, I'll provide a basic example of a word count program using MapReduce in Hadoop.

Environment Preparation

Before executing the program, ensure the following setup is complete:

- Hadoop Installation**

Hadoop should be properly installed and configured on your system.

- HDFS Configuration**

The Hadoop Distributed File System (HDFS) must be running, and input text files should be uploaded to HDFS.

Word Count Using MapReduce

1. Mapper Component

The Mapper is responsible for processing the input data. It reads the input text line by line, separates each line into individual words, and outputs each word paired with the value 1.

```
import java.io.IOException;
import org.apache.hadoop.io.IntWritable;
```



```
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;

public class WCMapper extends Mapper<LongWritable, Text, Text, IntWritable> {

    private static final IntWritable countOne = new IntWritable(1);
    private Text outputWord = new Text();

    @Override
    protected void map(LongWritable inputKey, Text inputValue, Context context)
        throws IOException, InterruptedException {

        String record = inputValue.toString();
        String[] tokenizedWords = record.split("\\s+");

        for (String token : tokenizedWords) {
            outputWord.set(token);
            context.write(outputWord, countOne);
        }
    }
}
```

2.Reducer Component

```
import java.io.IOException;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;

public class WCReducer extends Reducer<Text, IntWritable, Text, IntWritable> {

    @Override
    protected void reduce(Text inputKey, Iterable<IntWritable> inputValues, Context context)
        throws IOException, InterruptedException {
```

```
int totalCount = 0;

for (IntWritable val : inputValues) {
    totalCount += val.get();
}

context.write(inputKey, new IntWritable(totalCount));
}
}
```

3.Driver Component

```
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WCDriver {

    public static void main(String[] args) throws Exception {

        Configuration configuration = new Configuration();
        Job job = Job.getInstance(configuration, "Hadoop Word Count");

        job.setJarByClass(WCDriver.class);
        job.setMapperClass(WCMapper.class);
        job.setCombinerClass(WCReducer.class);
        job.setReducerClass(WCReducer.class);

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));
```

```
FileOutputFormat.setOutputPath(job, new Path(args[1]));  
  
System.exit(job.waitForCompletion(true) ? 0 : 1);  
}  
}
```

Executing the MapReduce Application

After writing the Mapper, Reducer, and Driver classes, the next step is to compile the code, store the input data in HDFS, and execute the MapReduce job on the Hadoop cluster.

Step 1: Compile Code and Create JAR File

First, compile all Java source files and generate class files using the Hadoop classpath. Then package the compiled classes into a JAR file.

```
javac -classpath $(hadoop classpath) -d wc_classes \  
WCMapper.java WCReducer.java WCDriver.java
```

Create the JAR file:

```
jar -cf wc_app.jar -C wc_classes/ .
```

Step 2: Upload Input Data to HDFS

Create a directory in HDFS to store the input files and upload the text file.

```
hdfs dfs -mkdir /wc_input  
hdfs dfs -put input.txt /wc_input
```

Step 3: Execute the MapReduce Job

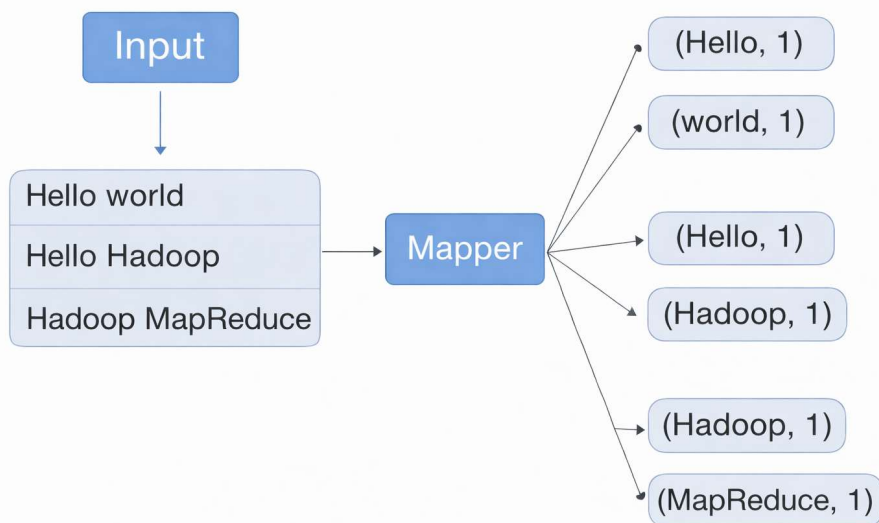
Run the MapReduce application by specifying the JAR file, Driver class, input directory, and output directory.

```
hadoop jar wc_app.jar WCDriver /wc_input /wc_output
```

Step 4: View the Output Results

Once the job finishes successfully, display the output file generated by the Reducer.

```
hdfs dfs -cat /wc_output/part-r-00000
```



CODE for program without Mapreduce:

```
import urllib.request

story = 'http://sixty-north.com/c/t.txt'
request = urllib.request.Request(story)
response = urllib.request.urlopen(request)

each_word = []
same_words = {}

# Collect all words
for line in response:
    line_words = line.split()
    for word in line_words:
```

```
each_word.append(word)
```

```
# Count words
```

```
for words in each_word:
```

```
    if words.lower() not in same_words:
```

```
        same_words[words.lower()] = 1
```

```
    else:
```

```
        same_words[words.lower()] += 1
```

```
# Print output
```

```
for each in same_words.keys():
```

```
    print("word = ", each, ", count = ", same_words[each])
```

```
word = b'it' , count = 10
word = b'was' , count = 11
word = b'the' , count = 14
word = b'best' , count = 1
word = b'of' , count = 12
word = b'times' , count = 2
word = b'worst' , count = 1
word = b'age' , count = 2
word = b'wisdom' , count = 1
word = b'foolishness' , count = 1
```

Practical -3

Aim: Basic CRUD Operations in MongoDB.

Program -1:

```
db.createCollection("staff")
```

```
db.staff.insertMany([  
  { empld: 101, name: "Rahul", dept: "IT" },  
  { empld: 102, name: "Sneha", dept: "HR" },  
  { empld: 103, name: "Amit", dept: "IT" }  
])
```

```
db.staff.find({ dept: "IT" })
```

```
db.staff.insertOne({ empld: 104, name: "Neha", dept: "Finance" })
```

```
db.staff.updateOne(  
  { name: "Neha" },  
  { $set: { name: "Neha Sharma" } }  
)
```

```
db.staff.find({ dept: "Finance" })
```

```
db.staff.deleteOne({ dept: "HR" })
```


Program -2 Create orders collection and items to the database.

```
db.createCollection("orders")

db.orders.insertMany( [

  { id: 0, name: "Pepperoni", size: "small", price: 19,
    quantity: 10, date: ISODate( "2021-03-13T08:14:30Z" ) },

  { id: 1, name: "Pepperoni", size: "medium", price: 20,
    quantity: 20, date : ISODate( "2021-03-13T09:13:24Z" ) },

  { id: 2, name: "Pepperoni", size: "large", price: 21,
    quantity: 30, date : ISODate( "2021-03-17T09:22:12Z" ) },

  { id: 3, name: "Cheese", size: "small", price: 12,
    quantity: 15, date : ISODate( "2021-03-13T11:21:39.736Z" ) },

  { id: 4, name: "Cheese", size: "medium", price: 13,
    quantity: 50, date : ISODate( "2022-01-12T21:23:13.331Z" ) },

  { id: 5, name: "Cheese", size: "large", price: 14,
    quantity: 10, date : ISODate( "2022-01-12T05:08:13Z" ) },

  { id: 6, name: "Vegan", size: "small", price: 17,
    quantity: 10, date : ISODate( "2021-01-13T05:08:13Z" ) },

  { id: 7, name: "Vegan", size: "medium", price: 18,
    quantity: 10, date : ISODate( "2021-01-13T05:10:13Z" ) }

] )

db.orders.find({size: "medium"});

db.orders.insert({id: 9, name: "Vegan", size: "medium", price: 8,
  quantity: 5, date : ISODate( "2021-01-22T05:10:13Z" ) })
```



```
db.orders.updateMany({name:'Vegan'},{$set:{name:'Veg'}}
db.orders.find({name: 'Veg'});
db.orders.remove({name:'Pepperoni'})
db.orders.find({ $and: [ {name: 'Veg'}, { size: "small"} ] })

db.orders.find()
```

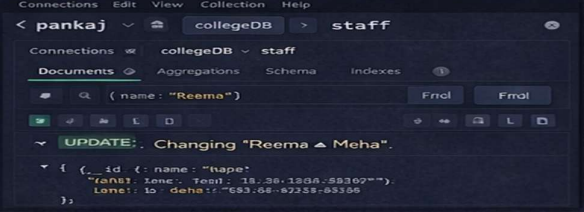
Insert, Update, Delete (CRUD Operations)

1. Staff Collection View



- **READ:** Showing all staff records in the "staff" collection

2. Update Reema to Meha



- **UPDATE:** Changing "Reema → Meha".
Result: 1 document updated

3. Delete Amit Employees



- **DELETE:** Removing all employees in "Amit" department.
Result: 2 documents deleted

Order Queries (Find Operations)

1. Find "Finance" Employees



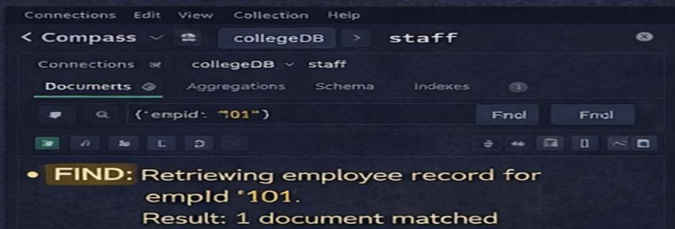
- **FIND:** Retrieving all employees in "Finance" department.
Result: 1 document matched

2. Find "Meha" (Previously Reema)



- **FIND:** Retrieving record where name is "Mehha".
Result: 1 document matched

3. Find Empld 101



- **FIND:** Retrieving employee record for empld '101'.
Result: 1 document matched

PRACTICAL – 4

Aim: Store the basic information about students such as roll no, name, Date of birth, and address of student using various collection types such as List, Set and Hadoop.

Code:

- Create database studentdb and Make Collection With name "student" and Follow Queries

✓ Created Database

> use studentdb

switched to DB studentdb

✓ Create Collection

> db.createCollection("student")

✓ Insert Records Into student Collection using **insertOne()** method

```
> db.student.insertOne({no:1, name:"ANUJ MULHAR",dob: "2004-01-30",e_mail: "thefunvlogs0@gmail.com", phone:"9904857092",address: {"B": "118","Nr.Kosamdi": "KUMKUM BUNGALOWS VALIA ROAD ANKLESHWAR","zipcode": "393001"},Branch:"CSE", marks:[58,96,55]})
```

Output:

```
db.student.insertOne({...})

{
  "acknowledged" : true,
  "insertedId" : ObjectId("65a1b2c3d4e5f67890123456")
}
```

Insert Records Into student Collection using **insertMany()** method.

```
db.student.insertMany([
  {no:2, name:"KUNAL NAKUM",dob: "2005-09-26",e_mail: "kunalnakum.123@gmail.com", phone:"9313564747",address:
  {"Kalam Bhawan-c ": "401","street": "WAGHODIA","zipcode":
  "391760"},Branch:"CSE", marks:[30,55,100]},
  {no:3, name:"Aditya patel",dob:
  "2002-10-21",e_mail: "adityapatel123@gmail.com",
  phone:"9313564747",address: {"Kalam Bhawan -C": "101","street":
  "Limda","zipcode": "391760"},Branch:"CSE", marks:[60,50,42]})])
```

Output:

```
db.student.insertMany([...])

{
  "acknowledged" : true,
  "insertedIds" : {
    "0" : ObjectId("65a1b2c3d4e5f67890123457"),
    "1" : ObjectId("65a1b2c3d4e5f67890123458")
  }
}
```

List out all Document

```
>db.student.find()
```

```
db.student.find()
```

```
{ "_id" : ObjectId("65a1b2c3d4e5f67890123456"),  
  "no" : 1,  
  "name" : "ANUJ MULHAR",  
  "dob" : "2004-01-30",  
  "e_mail" : "thefunvlogs0@gmail.com",  
  "phone" : "9904857092",  
  "Branch" : "CSE",  
  "marks" : [58, 96, 55]  
}  
  
{ "_id" : ObjectId("65a1b2c3d4e5f67890123457"),  
  "no" : 2,  
  "name" : "KUNAL NAKUM",  
  "dob" : "2005-09-26",  
  "e_mail" : "kunalnakum.123@gmail.com",  
  "phone" : "9313564747",  
  "Branch" : "CSE",  
  "marks" : [30, 55, 100]  
}  
  
{ "_id" : ObjectId("65a1b2c3d4e5f67890123458"),  
  "no" : 3,  
  "name" : "ADITYA PATEL",  
  "dob" : "2002-10-21",  
  "e_mail" : "adityapatel123@gmail.com",  
  "phone" : "9313564747"
```

update name of no 1 as "kunal sinh"

```
studentdb> db.student.update({no:1},{ $set:{name:"kunal sinh"}})
```

```
db.student.update({no:1}, {$set:{name:"kunal sinh"}})
```

```
WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
```

- ✓ To Find Document From the student collection where name begins with A

```
studentdb> db.student.find({name:/^A/})
```

```
db.student.find({name:/^A/})
```

```
{ "_id" : ObjectId("65a1b2c3d4e5f67890123458"),  
  "no" : 3,  
  "name" : "ADITYA PATEL",  
  "dob" : "2002-10-21",  
  "e_mail" : "adityapatel123@gmail.com",  
  "phone" : "9313564747",  
  "Branch" : "CSE",  
  "marks" : [60, 50, 42]  
}
```

Count:

It is used to count the number of document in the collections

```
db.student.count()
```

```
db.student.count()
```

```
3
```

limit() method is used to specify the maximum number of documents the cursor will return.

```
db.student.find().limit(2);
```

```
db.student.find().limit(2)
```

```
{ "_id" : ObjectId("65a1b2c3d4e5f67890123456"),  
  "no" : 1,  
  "name" : "kunal sinh",  
  "dob" : "2004-01-30",  
  "Branch" : "CSE",  
  "marks" : [58, 96, 55]  
}  
  
{ "_id" : ObjectId("65a1b2c3d4e5f67890123457"),  
  "no" : 2,  
  "name" : "KUNAL NAKUM",  
  "dob" : "2005-09-26",  
  "Branch" : "CSE",  
  "marks" : [30, 55, 100]  
}
```

set

Adds new fields to documents. \$set outputs documents that contain all existing fields from the input documents and newly added fields.

```
db.scores.insertMany([
```

```
{_id: 1, student: "Mulhar", homework: [ 10, 5, 10 ], quiz: [ 10, 8 ], extraCredit:  
0 },
```

```
{_id: 2, student: "ANUJ", homework: [ 5, 6, 5 ], quiz: [ 8, 8 ], extraCredit: 8 }}
```

```
db.scores.insertMany([
  { _id: 1, student: "Mulhar", homework: [10, 5, 10], quiz: [10, 8], extraCredit: 0 },
  { _id: 2, student: "ANUJ", homework: [5, 6, 5], quiz: [8, 8], extraCredit: 8 }
])

{
  "acknowledged" : true,
  "insertedIds" : {
    "0" : 1,
    "1" : 2
  }
}
```

- SET behavior, you should use \$addToSet, which avoids duplicates.

```
db.student.update(
  { no: 1 },
  { $addToSet: { marks: 96 } }
)
```

Output:

```
studentdb> db.student.update(
... { no: 1 },
... { $addToSet: { marks: 96 } }
)
WriteResult({ "nMatched" : 1, "nModified" : 1 })
```


Hadoop

Hadoop is used to store and process large student datasets in distributed manner using HDFS. MongoDB can act as a data source before transferring large data to Hadoop for analytics.

```
hdfs dfs -mkdir /student  
hdfs dfs -put studentdata.txt /student  
hdfs dfs -ls /student
```

Output:

```
$ hdfs dfs -ls /student  
Found 1 items  
-rw-r--r--  1 user supergroup laymra 2048 2024-04-24 10:30 /student/studentdata.txt
```

PRACTICAL-5

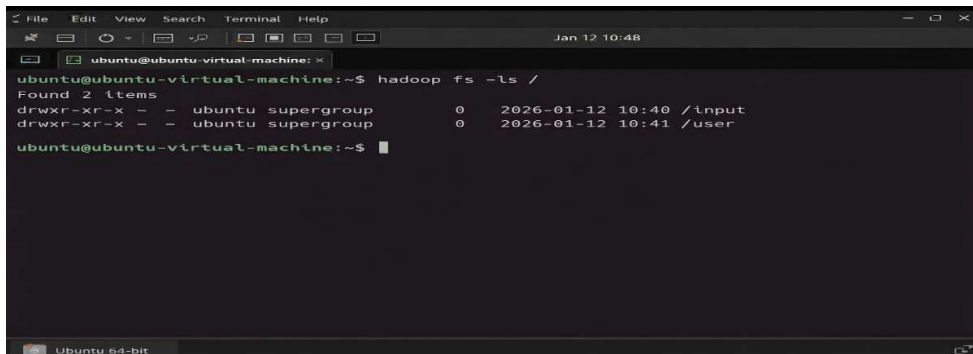
AIM: - Basic commands available for the Hadoop distributed file system.

HDFS provide a set of essential commands to interact with and manage files stored in the distributed environment. Here are some of the most commonly used commands.

File and Directory management

- **ls:** List the contents of a directory.

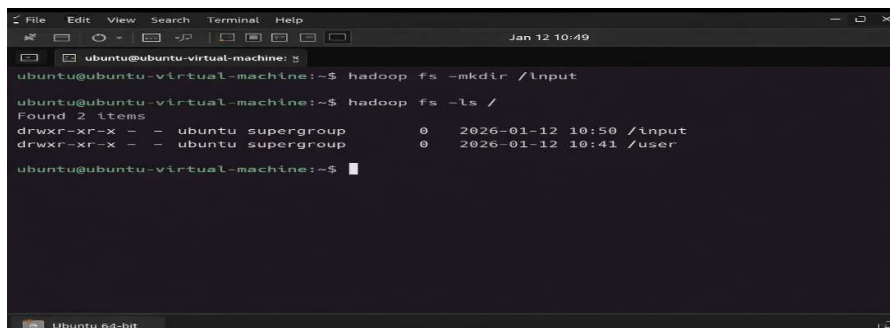
`hadoop fs -ls/path/to/directory`



```
ubuntu@ubuntu-virtual-machine:~$ hadoop fs -ls /
Found 2 items
drwxr-xr-x - - ubuntu supergroup          0   2026-01-12 10:40 /input
drwxr-xr-x - - ubuntu supergroup          0   2026-01-12 10:41 /user
ubuntu@ubuntu-virtual-machine:~$
```

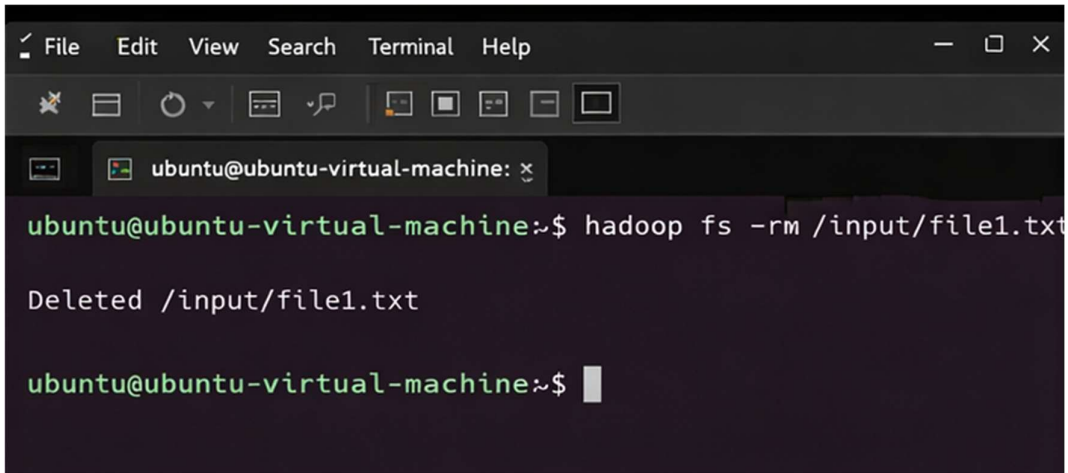
- **mkdir:** Creates a new directory.

`hadoop fs -mkdir/path/to/new/directory`



```
ubuntu@ubuntu-virtual-machine:~$ hadoop fs -mkdir /input
ubuntu@ubuntu-virtual-machine:~$ hadoop fs -ls /
Found 2 items
drwxr-xr-x - - ubuntu supergroup          0   2026-01-12 10:50 /input
drwxr-xr-x - - ubuntu supergroup          0   2026-01-12 10:41 /user
ubuntu@ubuntu-virtual-machine:~$
```

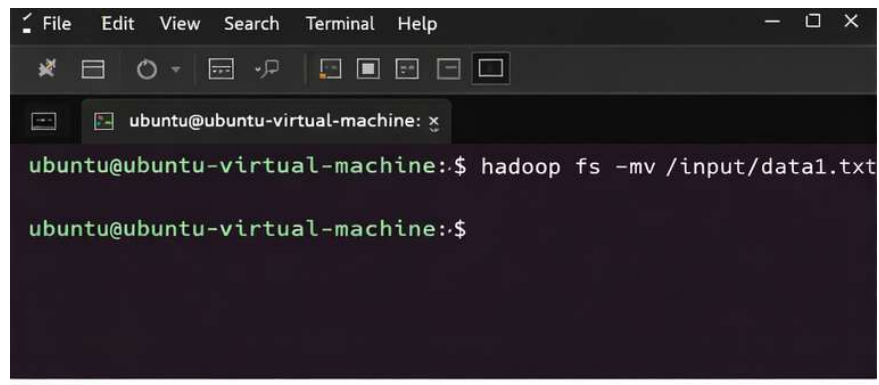
hadoop fs -rm /path/to/file



A screenshot of a terminal window titled 'ubuntu@ubuntu-virtual-machine: x'. The terminal shows the command 'hadoop fs -rm /input/file1.txt' being executed. The output is 'Deleted /input/file1.txt'. The prompt 'ubuntu@ubuntu-virtual-machine:~\$' is visible at the bottom.

- **mv:** Moves or rename a file or directory.

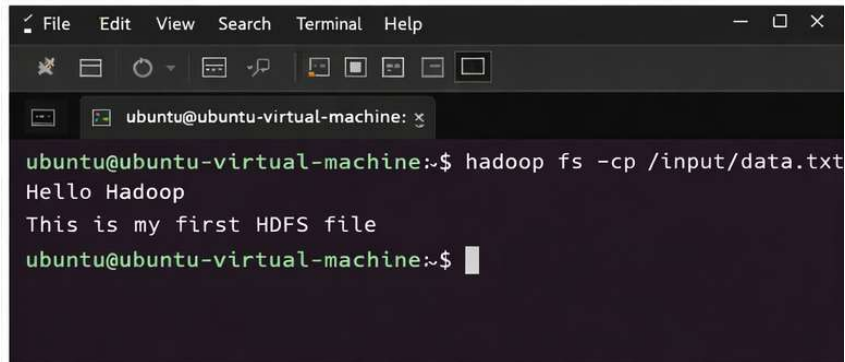
hadoop fs -mv /source/path/destination/path



A screenshot of a terminal window titled 'ubuntu@ubuntu-virtual-machine: x'. The terminal shows the command 'hadoop fs -mv /input/data1.txt' being executed. The output is 'ubuntu@ubuntu-virtual-machine:~\$'. The prompt 'ubuntu@ubuntu-virtual-machine:~\$' is visible at the bottom.

cp: Copies a file or directory.

hadoop fs -cp /source/path/destination/path



A terminal window titled 'ubuntu@ubuntu-virtual-machine' showing the execution of the command 'hadoop fs -cp /input/data.txt'. The output displays 'Hello Hadoop' and 'This is my first HDFS file'.

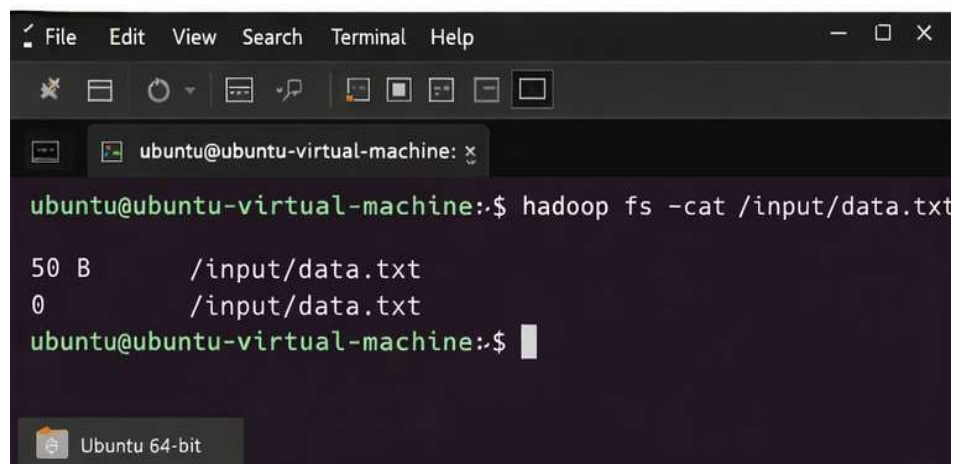
```
ubuntu@ubuntu-virtual-machine:~$ hadoop fs -cp /input/data.txt
Hello Hadoop
This is my first HDFS file
ubuntu@ubuntu-virtual-machine:~$
```

- **touchz:** Creates an empty file.

hadoop fs -touchz/path/to/file

- **cat:** display the content of a file

hadoop fs -cat/opath/to/file



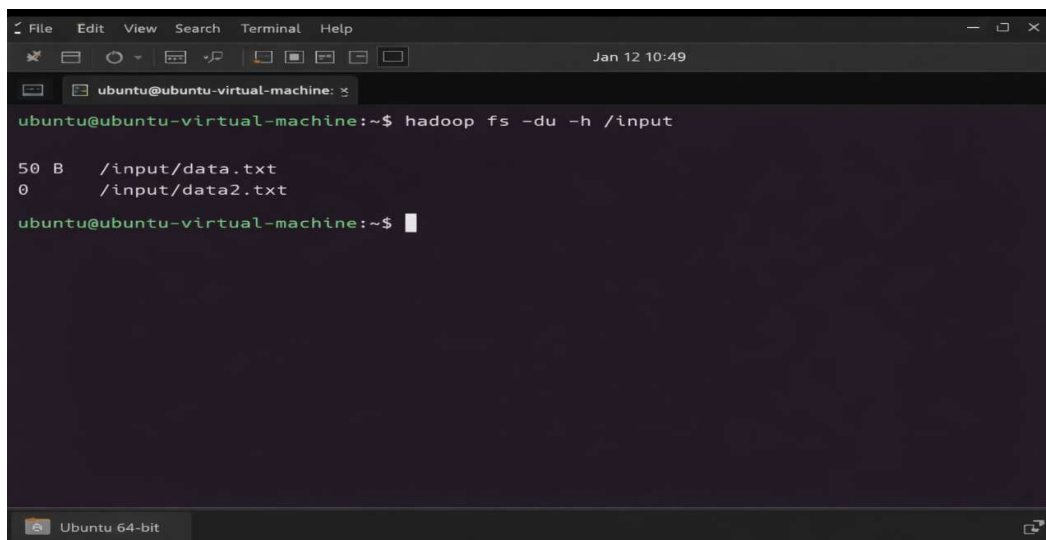
A terminal window titled 'ubuntu@ubuntu-virtual-machine' showing the execution of the command 'hadoop fs -cat /input/data.txt'. The output displays file statistics for '/input/data.txt'.

```
ubuntu@ubuntu-virtual-machine:~$ hadoop fs -cat /input/data.txt
50 B      /input/data.txt
0         /input/data.txt
ubuntu@ubuntu-virtual-machine:~$
```

`hadoop fs -cat/opath/to/file`

- **du:** Display disk usage statistics for a path

`hadoop fs -du -h/path/to/directory`



```
File Edit View Search Terminal Help
Jan 12 10:49
ubuntu@ubuntu-virtual-machine: ~$ hadoop fs -du -h /input
50 B  /input/data.txt
0     /input/data2.txt
ubuntu@ubuntu-virtual-machine: ~$
```

Data transfer

- **copyFromLocal:** Copies files from local file system to HDFS.

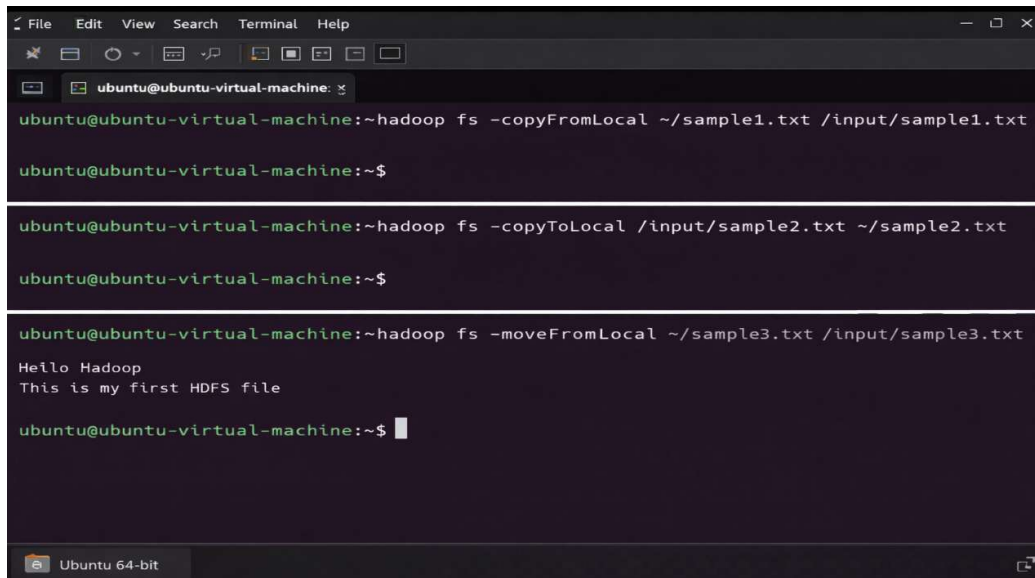
`hadoop fs -copyFromLocal/local/file/hdfs/destination`

- **copyToLocal:** Copies files from HDFS to local file system.

`hadoop fs -copyToLocal/hdfs/sources/local/destination`

- **moveFromLocal:** Moves files from local file system to HDFS.

`hadoop fs -moveFromLocal/local/file/hdfs/destination`



```
File Edit View Search Terminal Help
ubuntu@ubuntu-virtual-machine: ~$
ubuntu@ubuntu-virtual-machine:~$ hadoop fs -copyFromLocal ~/sample1.txt /input/sample1.txt
ubuntu@ubuntu-virtual-machine:~$
ubuntu@ubuntu-virtual-machine:~$ hadoop fs -copyToLocal /input/sample2.txt ~/sample2.txt
ubuntu@ubuntu-virtual-machine:~$
ubuntu@ubuntu-virtual-machine:~$ hadoop fs -moveFromLocal ~/sample3.txt /input/sample3.txt
Hello Hadoop
This is my first HDFS file
ubuntu@ubuntu-virtual-machine:~$
```

Other Useful Commands

- **df:** Displays HDFS filesystem status.

`hadoop fs -df -h`

- **chmod:** Changes file or directory permission.

`hadoop fs -chmod777/path/to/file`

- **chown:** Changes file or directory ownership.

`hadoop fs -chown user:group/path/to/file`

- **rm:** This command deletes a file from HDFS.

`hadoop fs -rm<filename/directoryName>`

- **du:** It will give the size of each file in directory.

`hadoop fs -du <dirName>`

- **dus:** This command will give the total size of directory/file.

hadoop fs -dus <dirName>

- **stat:** It will give the last modification time of directory or path. In short it will give stats of directory or file.

hadoop fs -stat <hdfs file>

- **setrep:** This command is used to change the replication factor of a file/directory in hdfs. by default, it is 3 for anything which is stored in HDFS (as set in hdfs coresite.xml).

- **test**

The test command is used for file test operations.

Options	Description
-d	Check whether the path given by the user is a directory or not, return 0 if it is a directory.
-e	Check whether the path given by the user exists or not, return if the path exists.
-f	Check whether the path given by the user is a file or not, return 0 if it is a file.
-s	Check if the path is not empty, return 0 if a path is not empty.
-r	Return 0 if the path exists and read permission is granted
-w	Return 0 if the exists and write permission is granted.
dfs	Checks whether the file size is 0 byte or not, return 0 if the file is of 0 bytes.

Practical -6

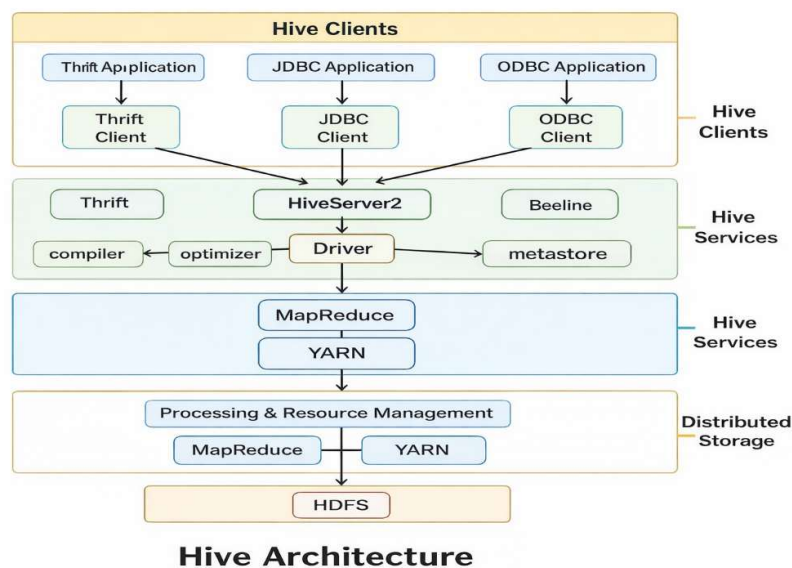
Aim: To study basic commands available for HIVE Query Language.

Description:

Apache Hive is an open-source data warehousing framework built on top of the Hadoop ecosystem. It is mainly used for analyzing and managing large datasets stored in the Hadoop Distributed File System (HDFS). Hive provides a query language known as Hive Query Language (HQL) which is similar to SQL, making it easy for users familiar with relational databases to work with big data. Hive converts HQL queries into distributed processing jobs such as MapReduce, Tez, or Spark that run on Hadoop clusters. This removes the need for developers to write complex MapReduce programs in Java. Using Hive, users can perform operations such as data summarization, analysis, and querying on large structured and semi-structured datasets.

Hive also supports connectivity with applications written in different programming languages like Java, Python, C++, and Ruby through standard interfaces such as JDBC, ODBC, and Thrift. These interfaces allow external applications to interact with Hive and execute queries on large data stored in Hadoop.

Overall, Hive simplifies big data analysis by providing an SQL-like interface for querying massive datasets efficiently.



Hive Architecture :

The architecture of Apache Hive consists of several layers that work together to process queries and store data efficiently.

1. Hive Clients

Users interact with Hive through different client applications such as:

- **Thrift Client**
- **JDBC Client**
- **ODBC Client**

These clients allow applications to send queries to the Hive system.

2. Hive Services

Hive services handle the processing of queries.

- **HiveServer2** – Accepts requests from different clients.
- **Driver** – Manages the execution of Hive queries.
- **Compiler** – Converts Hive queries into execution plans.
- **Optimizer** – Improves the performance of query execution.
- **Metastore** – Stores metadata such as table structure, schema, and database information.
- **Beeline** – Command-line interface used to interact with HiveServer2.

3. Processing Layer

The query execution is handled by Hadoop processing frameworks like:

- **MapReduce**
- **YARN (Yet Another Resource Negotiator)**

These frameworks process large datasets across distributed systems.

4. Storage Layer

At the bottom, data is stored in **HDFS (Hadoop Distributed File System)** which provides reliable distributed storage.

-- Step 1: Login to MySQL

```
mysql -u root -p
```

-- Step 2: Create Database

```
CREATE DATABASE school;
```

```
-- Step 3: Use Database  
USE school;
```

```
-- Step 4: Create Table  
CREATE TABLE teacher(  
ID INT PRIMARY KEY,  
Name VARCHAR(50),  
department VARCHAR(50),  
hiredate DATE,  
category VARCHAR(20),  
gender CHAR(1),  
salary INT  
);
```

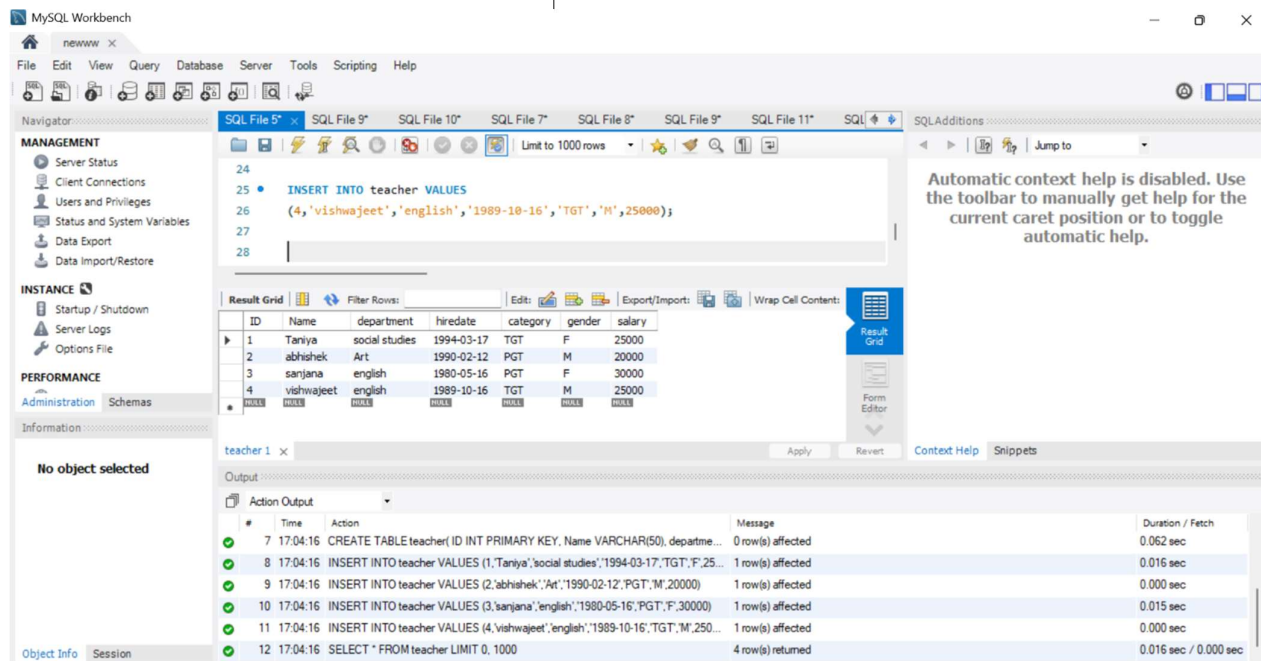
```
-- Step 5: Insert Data  
INSERT INTO teacher VALUES  
(1,'Taniya','social studies','1994-03-17','TGT','F',25000);
```

```
INSERT INTO teacher VALUES  
(2,'abhishek','Art','1990-02-12','PGT','M',20000);
```

```
INSERT INTO teacher VALUES  
(3,'sanjana','english','1980-05-16','PGT','F',30000);
```

```
INSERT INTO teacher VALUES  
(4,'vishwajeet','english','1989-10-16','TGT','M',25000);
```

```
-- Step 6: Display Data  
SELECT * FROM teacher;
```



The screenshot shows the MySQL Workbench interface. The query window contains the following SQL statement:

```
INSERT INTO teacher VALUES
(4, 'vishwajeet', 'english', '1989-10-16', 'TGT', 'M', 25000);
```

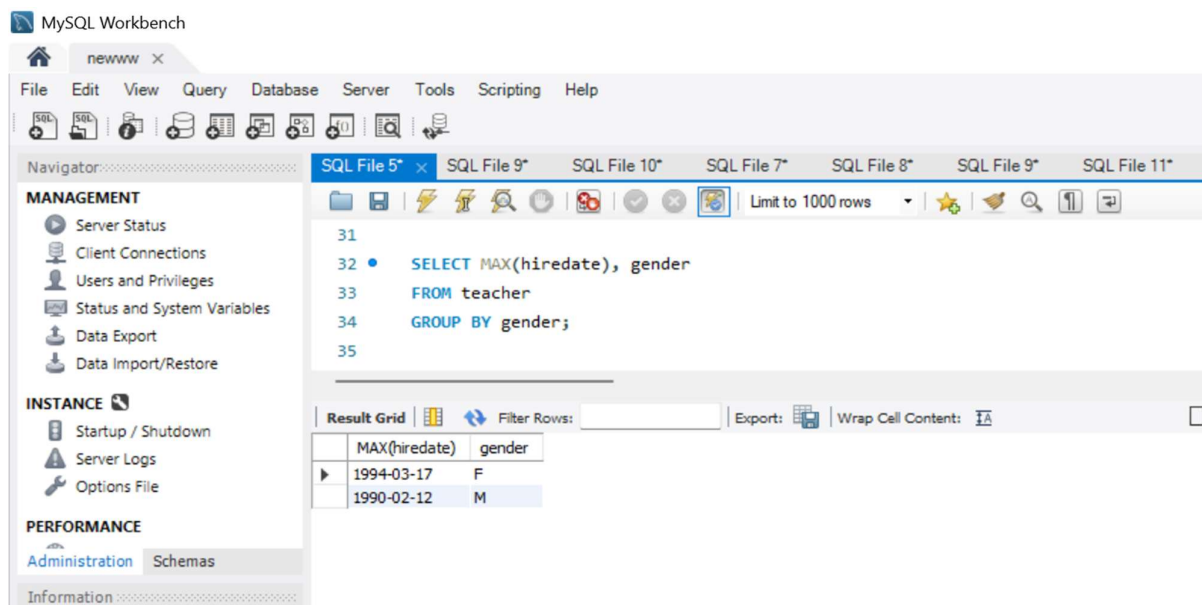
The result grid displays the following data:

ID	Name	department	hiredate	category	gender	salary
1	Taniya	social studies	1994-03-17	TGT	F	25000
2	abhishek	Art	1990-02-12	PGT	M	20000
3	sanjana	english	1980-05-16	PGT	F	30000
4	vishwajeet	english	1989-10-16	TGT	M	25000

The Action Output window shows the following results:

#	Time	Action	Message	Duration / Fetch
7	17:04:16	CREATE TABLE teacher(ID INT PRIMARY KEY, Name VARCHAR(50), departme...	0 row(s) affected	0.062 sec
8	17:04:16	INSERT INTO teacher VALUES (1,'Taniya','social studies','1994-03-17','TGT','F',25...	1 row(s) affected	0.016 sec
9	17:04:16	INSERT INTO teacher VALUES (2,'abhishek','Art','1990-02-12','PGT','M',20000)	1 row(s) affected	0.000 sec
10	17:04:16	INSERT INTO teacher VALUES (3,'sanjana','english','1980-05-16','PGT','F',30000)	1 row(s) affected	0.015 sec
11	17:04:16	INSERT INTO teacher VALUES (4,'vishwajeet','english','1989-10-16','TGT','M',250...	1 row(s) affected	0.000 sec
12	17:04:16	SELECT * FROM teacher LIMIT 0, 1000	4 row(s) returned	0.016 sec / 0.000 sec

- SELECT MAX(hiredate), gender
FROM teacher
GROUP BY gender;



The screenshot shows the MySQL Workbench interface. The query window contains the following SQL statement:

```
SELECT MAX(hiredate), gender
FROM teacher
GROUP BY gender;
```

The result grid displays the following data:

MAX(hiredate)	gender
1994-03-17	F
1990-02-12	M

PRACTICAL-7

AIM: Create Basic commands of HBASE Shell

Tools Required

- Hadoop
- HBase
- Linux Terminal

Introduction to Apache HBase

Apache HBase is a distributed, open-source NoSQL database that runs on top of the Hadoop Distributed File System (HDFS). It is designed to store and manage very large volumes of data while allowing fast and efficient read and write operations.

HBase follows a column-oriented data storage model instead of the traditional row-based model used in relational databases. It is inspired by Google's Bigtable and is widely used in big data environments where quick access to large datasets is required.

Because HBase works with Hadoop, it can scale easily across multiple machines and provide reliable data storage for large-scale applications.

Features of HBase

1. Distributed database that can run on many machines.
2. Supports real-time data read and write operations.
3. Built on Hadoop Distributed File System (HDFS).

4. Uses column-oriented data storage.
5. Capable of handling extremely large datasets.

HBase Data Model

The HBase data model consists of the following components:

1. Table

A table is the main structure used to store data in HBase.

2. Row Key

Each record in the table is identified by a unique row key.

3. Column Family

Columns are organized into groups known as column families.

Example:

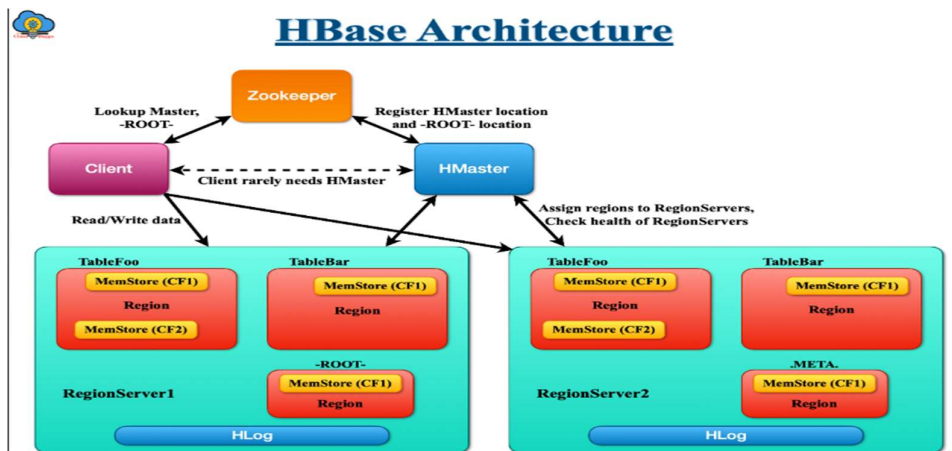
```
student
├── info:name
├── info:marks
```

4. Column Qualifier

A column qualifier represents a specific column inside a column family.

5. Cell

A cell is the smallest unit of data storage and contains the value associated with a row key and column.



HBase Architecture

1. HMaster

The HMaster server manages the HBase cluster and assigns regions to different region servers.

2. Region Server

Region servers are responsible for storing data and handling client read and write requests.

3. ZooKeeper

ZooKeeper helps coordinate communication between the distributed components of the HBase system.

4. HDFS

HDFS is the underlying storage system where all HBase data is physically stored.

HBase Shell

HBase provides a command-line interface known as the HBase Shell. It allows users to perform operations such as creating tables, inserting data, retrieving data, scanning tables, and deleting records.

Common commands include:

create
put
get
scan
delete
drop

Example command:

```
create 'student','info'
```

Advantages of HBase

- Efficient storage of large volumes of data
- Fast random read and write access
- Highly scalable architecture
- Reliable and fault-tolerant system

Applications of HBase

- Storage of social media data
- Processing and analyzing log data
- Real-time analytics systems
- Internet of Things (IoT) data storage

Procedure

Step 1 – Start HBase

start-hbase.sh

Step 2 – Open HBase Shell

hbase shell

Step 3 – Create Table

create 'student','info'

Step 4 – Insert Data

put 'student','29','info:name','shubham'

Step 5 – View Table Data

scan 'student'

Step 6 – Get Specific Row

get 'student','1'

Step 7 – Delete Data

delete 'student','1','info:name'

Step 8 – Drop Table

disable 'student'
drop 'student'

output:

```
[hadoop@localhost bin]$ start-hbase.sh
Starting HBase..
[hadoop@localhost bin]$ hbase shell
hbase(main):001:>
hbase(main):001:0> create 'student','info'
0 row(s) in 2.0710 seconds
=> Hbase::Table - student
hbase(main):002:0> put 'student', '29','info:name', 'shubham'
0 row(s) in 0.0389 seconds
hbase(main):003:0> get 'student', '29','info:name'
0 row(s) in 0.0200 seconds
hbase(main):006:0> disable 'student'
0 row(s) in 2.4529 seconds
```

PRACTICAL-8

AIM:Creating the HDFS tables and loading them in Hive and learning joining of tables in Hive.

Introduction to Hadoop

Hadoop is an open-source framework designed for storing and processing large volumes of data across distributed systems. It allows data to be stored across multiple machines while providing high reliability and scalability. Hadoop is widely used in big data environments where massive datasets must be processed efficiently.

The Hadoop ecosystem consists of several components that work together to manage and analyze big data. The most important component is the **Hadoop Distributed File System (HDFS)** which is responsible for storing large files across clusters of machines. Another important component is **Hive**, which provides an SQL-like interface for querying data stored in Hadoop.

Hadoop follows a distributed computing model where tasks are divided into smaller parts and executed on multiple nodes simultaneously. This helps in processing large datasets efficiently.

Hadoop Distributed File System (HDFS)

HDFS is the storage layer of the Hadoop ecosystem. It is designed to store very large files reliably across clusters of machines. HDFS divides large files into smaller blocks and distributes them across different nodes in a cluster.

The main components of HDFS are:

NameNode

The NameNode manages the metadata of the file system. It keeps track of file locations and manages access to files stored in the cluster.

DataNode

DataNodes are responsible for storing the actual data blocks. Each file stored in HDFS is split into multiple blocks and distributed across DataNodes.

Secondary NameNode

This node helps maintain the file system metadata and assists the NameNode by periodically merging edits with the file system image.

HDFS provides fault tolerance by replicating data blocks across multiple machines. If one machine fails, the data can still be accessed from another machine.

Introduction to Apache Hive

Apache Hive is a data warehouse tool built on top of Hadoop. It provides a SQL-like language called **Hive Query Language (HQL)** which allows users to query large datasets stored in HDFS without writing complex MapReduce programs.

Hive simplifies big data analysis by allowing users to perform operations such as:

- Data querying
- Data summarization
- Data analysis
- Table joins
- Data filtering and aggregation

Hive converts HQL queries into MapReduce or Spark jobs that run on the Hadoop cluster.

Because Hive supports SQL-like queries, it is widely used by data analysts who are familiar with relational database systems.

Hive Data Storage

Hive stores data in the form of tables. These tables are similar to tables in relational databases but the actual data is stored in HDFS.

Hive tables are categorized into two types:

Managed Tables

In managed tables, Hive manages both the table metadata and the actual data stored in HDFS. If the table is deleted, the data is also removed from HDFS.

External Tables

In external tables, Hive manages only the metadata. The actual data remains stored in HDFS even if the table is deleted.

Loading Data into Hive from HDFS

Hive allows data stored in HDFS to be loaded into tables for querying. This process involves creating a table structure in Hive and loading the data stored in HDFS files into the table.

The typical steps include:

1. Creating a table in Hive.
2. Loading data from HDFS into the table.

3. Querying the data using Hive commands.

Hive supports loading data using commands such as:

LOAD DATA INPATH

This command loads files stored in HDFS into a Hive table.

Joining Tables in Hive

Join operations are used to combine data from two or more tables based on a related column between them. Joins are commonly used in data analysis to combine information from multiple datasets.

Hive supports several types of join operations:

Inner Join

Returns records that have matching values in both tables.

Example concept:

Table A $\cap$ Table B

Left Outer Join

Returns all records from the left table and matching records from the right table. If no match is found, NULL values are returned.

Right Outer Join

Returns all records from the right table and matching records from the left table.

Full Outer Join

Returns all records from both tables whether there is a match or not.

Cross Join

Produces a Cartesian product of rows from both tables.

Joins in Hive help in combining datasets for analysis and extracting meaningful insights from large volumes of data.

Importance of Table Joins in Big Data

In big data environments, data is often stored in multiple tables or files. To perform meaningful analysis, it is necessary to combine data from different sources. Join operations make it possible to connect related datasets and analyze them together.

For example:

- Joining **student details table** with **marks table**
- Joining **customer information** with **transaction data**
- Combining **log files** with **user activity data**

Using joins, analysts can perform advanced queries and generate reports based on combined datasets.

Advantages of Using Hive with HDFS

Using Hive with HDFS provides several advantages:

- Ability to handle very large datasets
- SQL-like interface for querying big data

- Scalable distributed storage system
- Fault tolerance through data replication
- Efficient data processing using Hadoop framework

Hive makes big data analysis easier for users who are familiar with SQL.

Applications of Hive

Hive is widely used in big data applications such as:

- Data warehousing
- Business intelligence and reporting
- Log file analysis
- Social media data processing
- Large-scale data analytics

Many organizations use Hive to analyze large datasets and generate insights for decision making.

Step 1 — Start Hadoop and Hive

Open Linux terminal.

start-dfs.sh

start-yarn.sh

hive

- Hive shell will start

Step 2 — Create Database

CREATE DATABASE school;

SHOW DATABASES;

USE school;

Step 3 — Create Tables

Student Table

```
CREATE TABLE student(  
id INT,  
name STRING,  
dept STRING  
)  
ROW FORMAT DELIMITED  
FIELDS TERMINATED BY ',';
```

Marks Table

```
CREATE TABLE marks(  
id INT,  
subject STRING,  
marks INT  
)  
ROW FORMAT DELIMITED  
FIELDS TERMINATED BY ',';
```

Step 4 — Insert Data

Insert into student

```
INSERT INTO student VALUES  
(1,'Rahul','CS'),  
(2,'Anita','IT'),  
(3,'Karan','CS');
```

Insert into marks

```
INSERT INTO marks VALUES  
(1,'DBMS',80),  
(2,'Hadoop',75),  
(3,'Hive',85);
```


Step 5 — View Tables

```
SELECT * FROM student;
```

Output

1 Rahul CS

2 Anita IT

3 Karan CS

```
SELECT * FROM marks;
```

Step 6 — Perform JOIN in Hive

Inner Join

```
SELECT student.id, student.name, marks.subject, marks.marks
```

```
FROM student
```

```
JOIN marks
```

```
ON student.id = marks.id;
```

Step 7 — Left Join Example

```
SELECT student.name, marks.subject
```

```
FROM student
```

```
LEFT OUTER JOIN marks
```

```
ON student.id = marks.id;
```

id	name	subject	marks
1	Rahul	DBMS	80
2	Anita	Hadoop	75
3	Karan	Hive	85